

[08] Autoware Partition

1. Overview
2. Partitioning Approach
 - 2.1 컴포넌트 기반 파티셔닝
 - 장점
 - 단점
 - 2.2 패키지 기반 파티셔닝
 - 장점
 - 단점
3. 파티셔닝 구현 사례
 - 3.1 파티셔닝 전략 수립
 - 3.2 파티션 구성
 - 3.3 Docker Image 구성
 - 3.3.1 Dockerfile 정의
 - 3.3.2 의존성 해결
 - 3.4 시나리오 선택
 - 3.5 파티션별 Autoware 실행하기(시나리오 실행하기)
 - 3.5.1 파티션별 런처 구현
 - 3.5.2 의존성 해결
 - 3.5.2.1 의존성 문제의 종류와 해결 방안
 - 3.6 수정 내용 설명
4. 도커 이미지 빌드하기
 - 4.1 Autoware 다운로드
 - 4.2 이미지 빌드하기
 - 4.3 ARM 빌드 환경 설정하기
5. 개발 및 테스트 환경
 - 5.1 하드웨어 사양
 - 5.2 소프트웨어 환경
 - 5.3 시뮬레이션 환경
6. 테스트 방법

1. Overview

이 문서는 Autoware 프로젝트를 구성하는 다양한 모듈과 기능을 체계적으로 분리 및 구성하는 파티셔닝에 대해 다룹니다. Autoware는 자율주행 차량 개발을 위한 오픈소스 소프트웨어로, 다양한 센서 데이터 처리, 경로 계획, 제어 모듈 등이 복합적으로 연동되어 있습니다. 이러한 시스템을 효율적으로 관리하고 확장성을 확보하기 위해 각 모듈의 역할과 상호작용을 명확히 정의하고, 적절히 분리하는 작업은 필수적입니다.

이 문서의 주요 목적은 다음과 같습니다.

- **Autoware의 모듈화:** 각 모듈의 독립성을 보장하고 재사용성을 높이기 위한 파티셔닝 전략을 수립하는데에 기여하고자 합니다.
- **효율적인 시스템 설계:** 리소스 활용과 개발 효율성을 극대화할 수 있는 파티셔닝 방향을 설정하는데에 기여하고자 합니다.
- **기준 제공:** 개발자들이 시스템을 설계하고 수정할 때 참고할 수 있는 기준을 제공합니다.

파티셔닝의 방향성은 다음과 같은 사항들을 고려하여 설정됩니다.

- **기능 중심 분리:** 데이터 처리, 계획, 제어 등 주요 기능 단위로 모듈을 분리.
- **의존성 최소화:** 모듈 간 결합도를 낮추어 독립성을 유지.

- **확장성 확보:** 추가 기능 개발 시 최소한의 수정으로 연계 가능하도록 설계.
- **실시간성 고려:** 각 모듈의 성능 요구 사항을 충족하면서도 시스템의 실시간 처리 특성을 유지.

이후 챕터에서는 당사가 실제로 적용했던 파티셔닝 방법과 구현 과정에서의 고민, 테스트 결과를 구체적으로 기술할 것입니다. 이를 통해 Autoware를 체계적으로 나누고 설계하는 방법에 대한 실질적인 가이드를 제공하고자 합니다.

2. Partitioning Approach

Autoware의 파티셔닝은 시스템 구조와 개발 요구사항에 따라 다양한 방식으로 접근할 수 있습니다. 이 장에서는 당사가 파티셔닝의 feasibility 체크를 위해서 고려했던 아래의 두 가지 주요 방법에 대해서 설명합니다. 요약하자면, 이 두 가지 방법은 각각의 목적과 상황에 따라 적합성이 달라집니다. 컴포넌트 중심 파티셔닝은 구조적인 명확성과 확장성을 중시하는 프로젝트에 적합하며, 패키지 기반 파티셔닝은 리소스 절약과 실시간성을 우선시하는 환경에서 유리합니다.

2.1 컴포넌트 기반 파티셔닝

컴포넌트 기반 파티셔닝은 Autoware의 주요 하위 컴포넌트를 기준으로, 기능적으로 연관된 모듈들을 그룹화하여 파티션을 구성하는 방법입니다. 예를 들어, Perception과 Sensing 컴포넌트를 하나의 파티션으로 묶고, Localization과 Mapping 컴포넌트를 또 다른 파티션으로 구성하는 방식입니다.

장점

- 기능 중심 설계: 관련된 기능이 동일 파티션에 포함되므로 각 기능 내 연계가 용이함.
- 확장성과 재사용성: 특정 기능을 개선하거나 새로운 기능을 추가할 때, 다른 파티션에 미치는 영향 최소화.
- 디버깅 편의성: 모듈 간 데이터 흐름이 명확하므로 문제 발생 시 원인 추적이 용이함.

단점

- 복잡한 의존성: 컴포넌트 간의 의존성이 존재한다면, 의존성이 증가함.
- 리소스 최적화 어려움: 모든 컴포넌트를 동일 파티션에 포함할 경우, 불필요한 리소스 사용 발생함.
- 빌드 시간 증가 가능성: 파티션에 포함된 모든 패키지를 항상 빌드해야 하므로 시간과 리소스가 추가적으로 소모됨.

2.2 패키지 기반 파티셔닝

패키지 기반 파티셔닝은 파티션마다 필요한 패키지를 정확히 정의하고, 해당 패키지만 빌드하여 파티션을 구성하는 방법입니다. 각 파티션은 독립적으로 동작하며, 필요한 최소 패키지 세트를 포함합니다.

장점

- 효율적인 빌드: 필요한 패키지만 빌드하므로 시간과 리소스를 절약.
- 배포와 테스트 용이: 독립적인 패키지 세트를 포함하므로 특정 파티션만 테스트하거나 배포 가능함.
- 경량화 및 실시간성 강화: 각 파티션의 크기가 작아지고 실행 환경이 간소화되어 실시간 처리가 용이함.

단점

- 복잡성 증가: 각 파티션에 필요한 패키지를 정확히 식별하고 관리하는 데 추가적인 노력이 필요함.
- 유지보수 부담: 패키지 목록이 변경될 경우, 파티션 구성을 주기적으로 업데이트해야 함.
- 초기 설계 시간 증가: 모든 패키지의 의존성을 상세히 분석하고 파티션을 구성하는 데 시간이 더 소요됨.

3. 파티셔닝 구현 사례

이 장에서는 당사에서 실제로 Autoware 시스템에 파티셔닝을 적용했던 과정을 상세히 설명합니다. 앞서 제시한 두 가지 파티셔닝 방법론 중에 컴포넌트 기반 파티셔닝을 바탕으로 호스트 PC 환경에서 구현하고 검증한 사례를 다룹니다.

적용 과정에서 직면한 설계상의 고민과 해결책을 공유함으로써 파티셔닝 설계에 최대한 도움이 되고자 합니다.

이 장의 내용을 통해 Autoware 파티셔닝을 효과적으로 설계하고 구현하기 위한 실질적인 가이드라인을 제공하는 것을 목표로 합니다.

3.1 파티셔닝 전략 수립

효율적인 Autoware 시스템 분리를 위해 당사는 **컴포넌트 기반 파티셔닝**을 선택하고, 이를 **Docker**를 활용해 구현하기로 했습니다. 아래에서 이러한 전략을 수립하게 된 배경에 대해서 설명합니다.

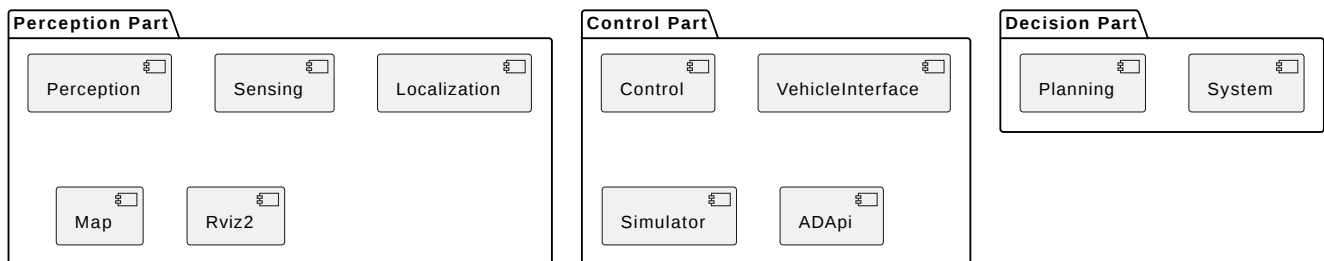
- **패키지 기반 파티셔닝 진행의 걸림돌**
 - Autoware의 패키지 분석에 상당한 시간이 소요될 것으로 예상.
 - 어떤 시나리오를 기준으로 feasibility 체크를 해야 할지 명확히 결정되지 않은 상태.
 - 정상적으로 동작되는 시나리오가 존재할지 불분명하여, 초기 테스트에 높은 불확실성이 존재.
- **컴포넌트 기반 파티셔닝의 장점**
 - 주요 기능(Perception, Localization, Control 등)을 기반으로 파티션을 나눌 수 있어 구조적으로 명확하고 설계 및 관리가 용이.
 - 패키지 분석 및 상세 의존성 검토 없이도 파티셔닝이 가능하여 초기 단계에서의 불확실성을 줄임.
 - 테스트 시나리오가 변경되더라도 즉각적으로 대응 가능함.
 - 일정 내에 파티셔닝 진행이 가능함.
- **Docker 활용**
 - 독립적인 하드웨어가 없는 상태에서도 Docker 컨테이너를 활용해 각 파티션을 독립적으로 실행 가능.
 - 추가 컴포넌트를 컨테이너로 손쉽게 확장할 수 있어, 이후 시스템 확장에 유연하게 대응 가능.

위와 같은 이유로 컴포넌트 기반 파티셔닝과 Docker를 활용한 접근은 초기 불확실성을 최소화하고, 독립적인 테스트 및 개발 환경을 구축하기에 적합한 전략이며, 초기 개발 단계에서의 효율성을 극대화하고, 이후 하드웨어와 통합할 때에도 상당한 유연성을 보장할 수 있을 것으로 판단했습니다.

3.2 파티션 구성

기본적인 전략이 수립되었다면, 실제로 각 파티션을 정의하고 구성하는 단계로 들어갑니다. 파티션 구성은 시스템 구조와 개발 요구사항, 하드웨어 구조 등 여러 요소를 고려하여 결정해야 합니다.

당사는 AD SW에 실제 적용될 구성을 참고하여 아래와 같이 3파트로 파티션을 구성합니다. 기능 연관성과 컴포넌트 간의 의존성, 그리고 로드 밸런싱에 중점을 두고 구성하였습니다.



3.3 Docker Image 구성

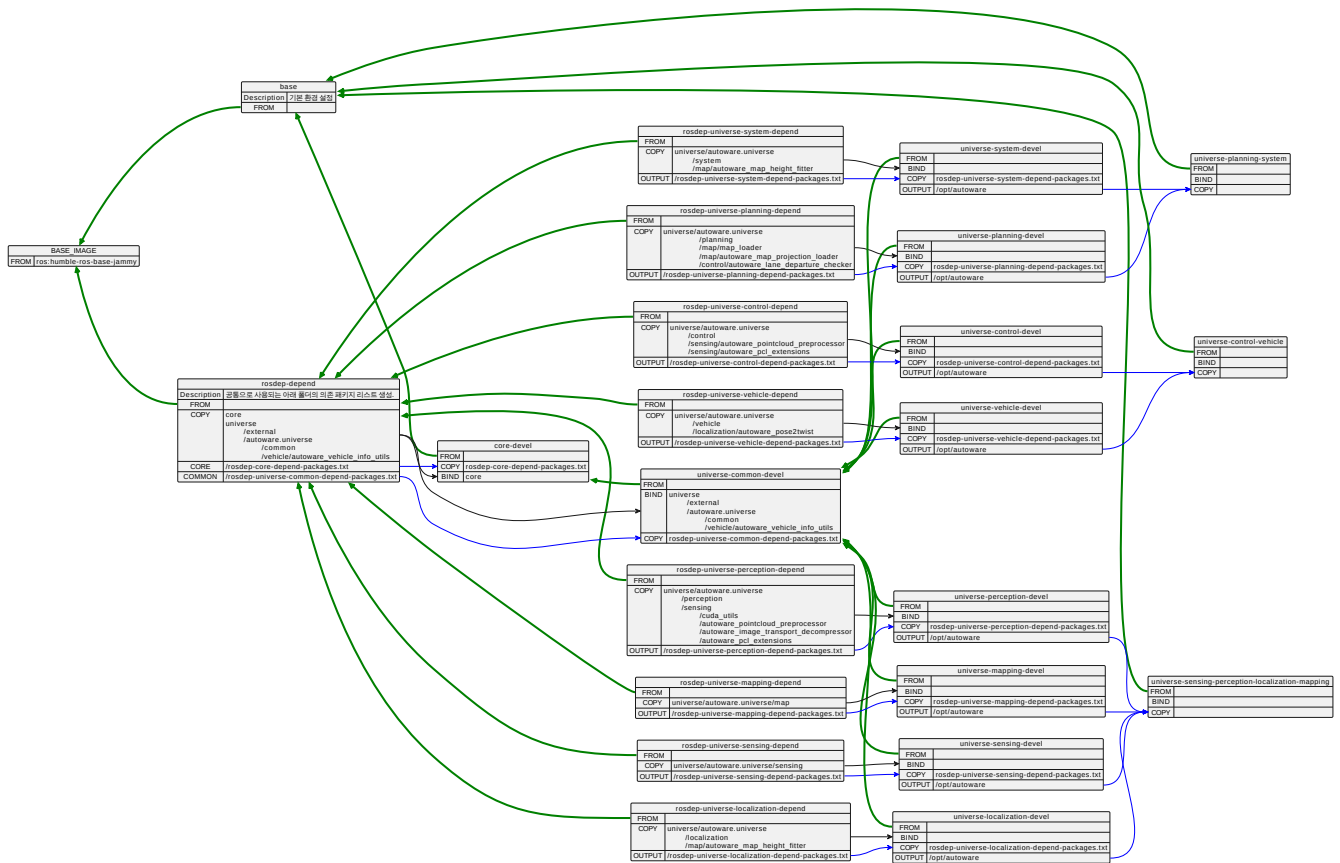
파티션 구성이 끝났다면 이제 구성된 대로 파티션이 생성되도록 필요한 소프트웨어와 패키지를 Dockerfile로 정의해야 합니다. 당사는 이 과정에 있어서 아래와 같은 원칙을 세우고 Dockerfile을 정의하였습니다.

- 공통으로 사용되는 부분은 별도의 이미지로 정의하여 재사용성을 높이고 이미지 빌드 시간을 단축시킨다.
- 각각의 컴포넌트 별로 이미지를 정의하여 파티션 구성을 자유롭게 할 수 있도록 유연성을 확보한다.

3.3.1 Dockerfile 정의

다음은 Dockerfile을 정의하기 위한 절차와 설명입니다. 지면상이라 설명이 너무 장황해 질 수가 있어 이해를 돕기 위해 필요한 부분만 설명합니다. 자세한 부분은 당사에서 제공한 소스 코드의 Dockerfile을 참고하시기 바랍니다.

다음 절차를 따라가면 최종적으로는 아래의 구조를 가지는 Dockerfile이 정의가 됩니다. 컴포넌트당 하나의 이미지를 구성해 놓았기 때문에 원하는 대로 조합하여 최종 이미지를 구성할 수 있습니다.



1. 기본 이미지 정의

- Autoware는 ROS2 기반이라 ros-humble-ros-base-jammy 이미지를 기준 이미지로 사용합니다.
- 이 기준 이미지에 아래에 정의된 작업을 추가하여 기본 이미지를 정의합니다.
- 기본 이미지는 모든 컴포넌트의 기준 이미지가 됩니다.

```
1 FROM $BASE_IMAGE AS base
2 # Install apt packages and add GitHub to known hosts for private repositories
3 ...
4 # Copy files
5 COPY setup-dev-env.sh ansible-galaxy-requirements.yaml amd64.env arm64.env /autoware/
6 COPY ansible/ /autoware/ansible/
7 COPY src/launcher/autoware_launch/autoware_launch/config/ /autoware_config/
8 WORKDIR /autoware
9 # Set up base environment
10 ...
11 # Create entrypoint
12 ...
```

2. Common 컴포넌트 정의

- Autoware에서 공통으로 사용되는 부분입니다.
- 아래의 모든 컴포넌트의 기준 이미지가 됩니다.

```
1 FROM $BASE_IMAGE AS rosdep-depend
2 ...
3 # Generate install package lists
```

```

4 COPY src/core /autoware/src/core
5 COPY src/universe/external /autoware/src/universe/external
6 COPY src/universe/autoware.universe/common
  /autoware/src/universe/autoware.universe/common
7 RUN rosdep keys --ignore-src --from-paths src \
8   | xargs rosdep resolve --rosdistro ${ROS_DISTRO} \
9   | grep -v '^#' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-common-depend-
  packages.txt \
10   && cat /rosdep-universe-common-depend-packages.txt

```

```

1 FROM core-devel AS universe-common-devel
2 ...
3 # Install rosdep dependencies
4 COPY --from=rosdep-depend /rosdep-universe-common-depend-packages.txt /tmp/rosdep-
  universe-common-depend-packages.txt
5 RUN --mount=type=ssh apt-get update \
6   && cat /tmp/rosdep-universe-common-depend-packages.txt | xargs apt-get install -y
  --no-install-recommends \
7   && apt-get autoremove -y && apt-get clean -y && rm -rf /var/lib/apt/lists/*
  "$HOME"/.cache
8
9 RUN --mount=type=cache,target=${CCACHE_DIR} \
10  --mount=type=cache,target=${CCACHE_DIR} \
11  --mount=type=bind,from=rosdep-
  depend,source=/autoware/src/core,target=/autoware/src/core \
12  --mount=type=bind,from=rosdep-
  depend,source=/autoware/src/universe/autoware.universe/common,target=/autoware/src/u
  niverse/autoware.universe/common \
13  --mount=type=bind,from=rosdep-
  depend,source=/autoware/src/universe/external,target=/autoware/src/universe/external
  \
14  source /opt/ros/"$ROS_DISTRO"/setup.bash && source /opt/autoware/setup.bash \
15  && du -sh ${CCACHE_DIR} && ccache -s \
16  && colcon build --cmake-args " -Wno-dev" " --no-warn-unused-cli" --merge-install \
17  --install-base /opt/autoware --mixin release compile-commands ccache \
18  && du -sh ${CCACHE_DIR} && ccache -s && rm -rf /autoware/build /autoware/log
19

```

i 모든 컴포넌트(Common, Perception, Sensing...)는 위와 같이 2개의 스테이지(FROM)가 하나의 쌍으로 정의된다.

- 해당 컴포넌트의 ROS 의존성 패키지 리스트를 작성하는 부분
- 작성된 리스트를 참고하여 의존성 패키지를 설치하고 해당 컴포넌트를 빌드하는 부분

i 구성 요소

- src/core
- src/universe/external
- src/universe/autoware.universe/common

i 파티셔닝시 모든 파티션에 공통적으로 들어갈 패키지가 있다면, Common 컴포넌트에 정의하면 된다.

3. Perception 컴포넌트 정의

- Common 컴포넌트를 기준 이미지로 사용한다.

```

1 FROM rosdep-depend AS rosdep-universe-perception-depend
2 ...

```

```

3 COPY src/universe/autoware.universe/perception
  /autoware/src/universe/autoware.universe/perception
4
5 RUN rosdep keys --ignore-src --from-paths src \
6   | xargs rosdep resolve --rosdistro ${ROS_DISTRO} \
7   | grep -v '^#' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-perception-depend-
  packages.txt \
8   && cat /rosdep-universe-perception-depend-packages.txt
9 RUN rosdep keys --dependency-types=exec --ignore-src --from-paths src \
10  | xargs rosdep resolve --rosdistro ${ROS_DISTRO} \
11  | grep -v '^#' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-perception-exec-
  depend-packages.txt \
12  && cat /rosdep-universe-perception-exec-depend-packages.txt

```

```

1 FROM universe-common-devel AS universe-perception-devel
2 ...
3 # Install rosdep dependencies
4 COPY --from=rosdep-universe-perception-depend /rosdep-universe-perception-depend-
  packages.txt /tmp/rosdep-universe-perception-depend-packages.txt
5 # hadolint ignore=SC2002
6 RUN --mount=type=ssh \
7   apt-get update \
8   && cat /tmp/rosdep-universe-perception-depend-packages.txt | xargs apt-get install
  -y --no-install-recommends \
9   && apt-get autoremove -y && apt-get clean -y && rm -rf /var/lib/apt/lists/*
  "$HOME"/.cache
10 # build perception
11 RUN --mount=type=cache,target=${CCACHE_DIR} \
12   --mount=type=bind,from=rosdep-universe-perception-
  depend,source=/autoware/src/universe/autoware.universe/perception,target=/autoware/s
  rc/universe/autoware.universe/perception \
13   source /opt/ros/"${ROS_DISTRO}"/setup.bash && source /opt/autoware/setup.bash \
14   && du -sh ${CCACHE_DIR} && ccache -s \
15   && colcon build --cmake-args " -Wno-dev " --no-warn-unused-cli --merge-install \
16   --install-base /opt/autoware --mixin release compile-commands ccache \
17   && du -sh ${CCACHE_DIR} && ccache -s && rm -rf /autoware/build /autoware/log
18 ...

```

구성 요소

- src/universe/autoware.universe/perception

그 외 설명은 'Common 컴포넌트 정의' 참고.

5. Sensing/Localization/Mapping/Planning/Control/VehicleInterface/System/Simulator/ADApi

각각의 컴포넌트 구성 요소

- src/universe/autoware.universe/control
- src/universe/autoware.universe/localization
- src/universe/autoware.universe/map
- src/universe/autoware.universe/planning
- src/universe/autoware.universe/sensing
- src/universe/autoware.universe/simulator
- src/universe/autoware.universe/system
- src/universe/autoware.universe/vehicle

- src/universe/autoware.universe/system/autoware_default_adapi
- src/universe/autoware.universe/system/default_ad_api_helpers/ad_api_adaptors

그 외 설명은 'Perception 컴포넌트 정의' 참고.

6. Perception part 정의

- Perception part에 들어갈 컴포넌트들의 의존성 패키지 리스트를 복사하여 환경 설정을 한다.
- Perception part에 들어갈 컴포넌트들의 빌드된 패키지를 복사한다.

```

1 FROM base AS adsw-perception
2 ...
3 # Set up runtime environment and artifacts
4 COPY --from=rosdep-universe-perception-depend /rosdep-universe-perception-exec-depend-packages.txt
  /tmp/perception-depend.txt
5 COPY --from=rosdep-universe-sensing-depend /rosdep-universe-sensing-exec-depend-packages.txt /tmp/sensing-
  depend.txt
6 COPY --from=rosdep-universe-localization-depend /rosdep-universe-localization-exec-depend-packages.txt
  /tmp/localization-depend.txt
7 COPY --from=rosdep-universe-mapping-depend /rosdep-universe-mapping-exec-depend-packages.txt /tmp/mapping-
  depend.txt
8
9 RUN cat /tmp/perception-depend.txt /tmp/sensing-depend.txt /tmp/localization-depend.txt /tmp/mapping-
  depend.txt | sort | uniq > /tmp/depend.txt
10 RUN --mount=type=ssh \
11     --mount=type=cache,target=/var/cache/apt,sharing=locked \
12     ./setup-dev-env.sh -y --module all ${SETUP_ARGS} --download-artifacts --no-cuda-drivers --runtime openadkit
  \
13     && pip uninstall -y ansible ansible-core \
14     && apt-get update \
15     && cat /tmp/depend.txt | xargs apt-get install -y --no-install-recommends \
16     ...
17
18 COPY --from=universe-perception-devel /opt/autoware /opt/autoware
19 COPY --from=universe-sensing-devel /opt/autoware /opt/autoware
20 COPY --from=universe-localization-devel /opt/autoware /opt/autoware
21 COPY --from=universe-mapping-devel /opt/autoware /opt/autoware
22 ...

```

7. Decision/Control part 정의

- 'Perception part 정의' 참고

3.3.2 의존성 해결

위와 같이 Dockerfile을 작성하고, 다음으로는 의존성 문제를 해결해야 합니다. Autoware의 각 패키지들은 의존성 정보를 'package.xml' 파일에 작성을 하는데, colcon 빌드 시스템은 해당 정보를 분석하여 의존 패키지가 존재하지 않으면 아래와 같이 빌드 실패를 발생시킵니다.

```
zeus@zeus-Super-PC: ~/test/autoware
zeus@zeus-Su... root@zeus-Su... root@zeus-Su... root@zeus-Su... root@zeus-Su... zeus@zeus-Su... zeus@zeus-Su...
# ' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-common-depend-packages.txt && cat /rosdep-universe-common-depend-packa
ges.txt:
0.726 ERROR: no rosdep rule for 'tier4_autoware_api_launch'
0.726 ERROR: no rosdep rule for 'tier4_sensing_launch'
0.726 ERROR: no rosdep rule for 'tier4_control_launch'
0.726 ERROR: no rosdep rule for 'ad_api_adaptors'
0.726 ERROR: no rosdep rule for 'tier4_localization_launch'
0.726 ERROR: no rosdep rule for 'tier4_vehicle_launch'
0.726 ERROR: no rosdep rule for 'tier4_system_launch'
0.726 ERROR: no rosdep rule for 'tier4_map_launch'
0.726 ERROR: no rosdep rule for 'tier4_perception_launch'
0.726 ERROR: no rosdep rule for 'tier4_simulator_launch'
-----
Dockerfile:74
-----
73 |
74 | >>> RUN rosdep keys --ignore-src --from-paths src \
75 | >>> | xargs rosdep resolve --rosdistro ${ROS_DISTRO} \
76 | >>> | grep -v '^#' \
77 | >>> | sed 's/ \+/\\n/g' \
78 | >>> | sort \
79 | >>> > /rosdep-universe-common-depend-packages.txt \
80 | >>> && cat /rosdep-universe-common-depend-packages.txt
81 |
-----
ERROR: failed to solve: process "/bin/bash -o pipefail -c rosdep keys --ignore-src --from-paths src | xargs rosdep resolve --rosd
istro ${ROS_DISTRO} | grep -v '^#' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-common-depend-packages.txt && ca
t /rosdep-universe-common-depend-packages.txt" did not complete successfully: exit code: 123
```

의존성 문제를 해결하는 방법은 아래 방법중 적당한 방법을 선택하여 해결합니다.

- 의존 패키지를 파티션에 추가하여 해결
 - colcon 빌드 시스템의 여러 메시지를 통해 의존 패키지를 확인할 수 있어 관련 패키지를 추가해 가면서 문제를 해결한다.(당사가 진행했던 컴포넌트 기반 파티셔닝에 적합)
 - colcon 빌드 시스템의 '--packages-up-to' 옵션을 사용하여 해당 패키지의 의존 패키지를 사전에 파악하여 도커 이미지를 구성한다.(패키지 기반 파티셔닝에 적합)
- 의존성을 제거하여 해결
 - 해당 패키지의 의존성을 'package.xml' 파일에서 제거한다. Autoware의 기본 소스를 수정해야하는 부담감도 있고, 당사의 feasibility 체크 업무와 무관하여 고려하지 않았습니다.

i 의존 패키지를 추가하는 것은 파티셔닝의 독립성과 경량화를 해치게 될 수 있으므로 신중한 검토가 필요합니다. 'Autoware 실행하기'에서 자세히 다루겠습니다.

이 단계의 의존성 해결은 이미지 빌드가 목표입니다. 사전에 충분한 검토가 이루어졌다면 런처 구성등을 함께 고려하여 의존성을 해결할 수 있습니다.

아래는 perception 컴포넌트의 의존성 문제를 해결한 Dockerfile 정의 내용입니다. 당사는 perception 외에 몇몇 의존 패키지를 추가하여 의존성 문제를 해결하였습니다. 그 외 컴포넌트들도 아래와 동일한 방법으로 의존성을 해결합니다.

```
1 FROM rosdep-depend AS rosdep-universe-perception-depend
2 ...
3 COPY src/universe/autoware.universe/perception
  /autoware/src/universe/autoware.universe/perception
4 COPY src/universe/autoware.universe/sensing/autoware_cuda_utils
  /autoware/src/universe/autoware.universe/sensing/autoware_cuda_utils
5 COPY src/universe/autoware.universe/sensing/autoware_pointcloud_preprocessor
  /autoware/src/universe/autoware.universe/sensing/autoware_pointcloud_preprocessor
6 COPY src/universe/autoware.universe/sensing/autoware_image_transport_decompressor
  /autoware/src/universe/autoware.universe/sensing/autoware_image_transport_decompress
  or
7 COPY src/universe/autoware.universe/sensing/autoware_pcl_extensions
  /autoware/src/universe/autoware.universe/sensing/autoware_pcl_extensions
8
9 RUN rosdep keys --ignore-src --from-paths src \
10 | xargs rosdep resolve --rosdistro ${ROS_DISTRO} \
11 | grep -v '^#' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-perception-depend-
  packages.txt \
```



```

12    && cat /rosdep-universe-perception-depend-packages.txt
13 RUN rosdep keys --dependency-types=exec --ignore-src --from-paths src \
14     | xargs rosdep resolve --rosdistro ${ROS_DISTRO} \
15     | grep -v '^#' | sed 's/ \+/\\n/g' | sort > /rosdep-universe-perception-exec-
depend-packages.txt \
16    && cat /rosdep-universe-perception-exec-depend-packages.txt

```

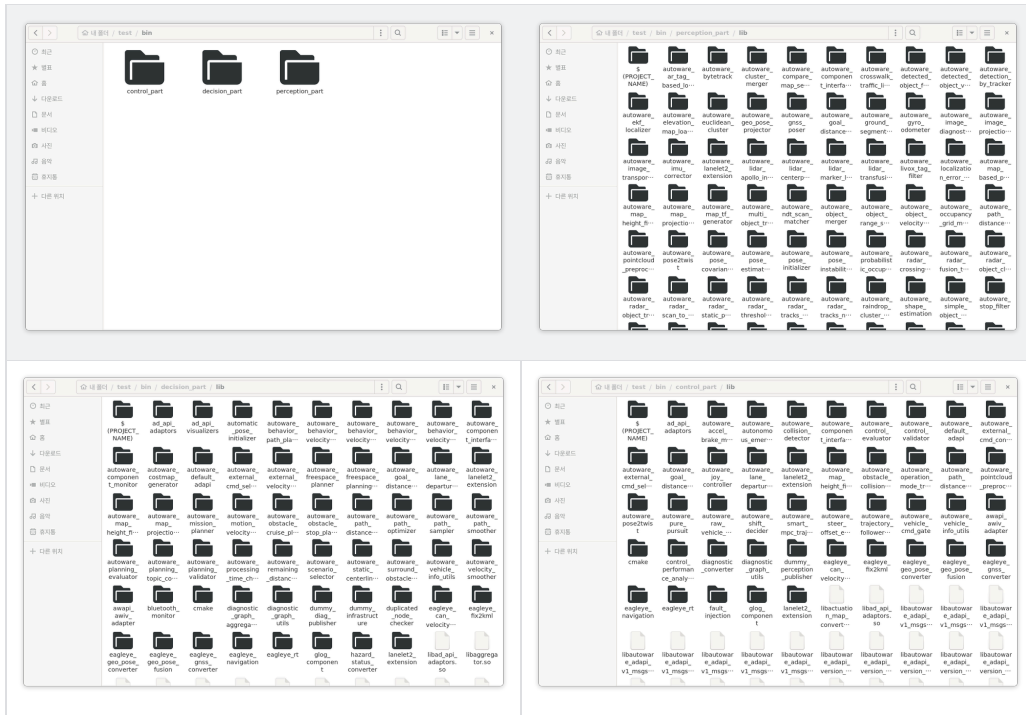
```

1 FROM universe-common-devel AS universe-perception-devel
2 ...
3 # Install rosdep dependencies
4 COPY --from=rosdep-universe-perception-depend /rosdep-universe-perception-depend-
packages.txt /tmp/rosdep-universe-perception-depend-packages.txt
5 RUN --mount=type=ssh \
6     apt-get update \
7     && cat /tmp/rosdep-universe-perception-depend-packages.txt | xargs apt-get install
-y --no-install-recommends \
8     && apt-get autoremove -y && apt-get clean -y && rm -rf /var/lib/apt/lists/*
"$HOME"/.cache
9 # build perception
10 RUN --mount=type=cache,target=${CCACHE_DIR} \
11     --mount=type=bind,from=rosdep-universe-perception-
depend,source=/autoware/src/universe/autoware.universe/perception,target=/autoware/s
rc/universe/autoware.universe/perception \
12     --mount=type=bind,from=rosdep-universe-perception-
depend,source=/autoware/src/universe/autoware.universe/sensing/autoware_cuda_utils,t
arget=/autoware/src/universe/autoware.universe/sensing/autoware_cuda_utils \
13     --mount=type=bind,from=rosdep-universe-perception-
depend,source=/autoware/src/universe/autoware.universe/sensing/autoware_pointcloud_p
reprocessor,target=/autoware/src/universe/autoware.universe/sensing/autoware_pointcl
oud_preprocessor \
14     --mount=type=bind,from=rosdep-universe-perception-
depend,source=/autoware/src/universe/autoware.universe/sensing/autoware_image_transp
ort_decompressor,target=/autoware/src/universe/autoware.universe/sensing/autoware_im
age_transport_decompressor \
15     --mount=type=bind,from=rosdep-universe-perception-
depend,source=/autoware/src/universe/autoware.universe/sensing/autoware_pcl_extensio
ns,target=/autoware/src/universe/autoware.universe/sensing/autoware_pcl_extensions \
16     source /opt/ros/"$ROS_DISTRO"/setup.bash && source /opt/autoware/setup.bash \
17     && du -sh ${CCACHE_DIR} && ccache -s \
18     && colcon build --cmake-args " -Wno-dev" " --no-warn-unused-cli" --merge-install \
19     --install-base /opt/autoware --mixin release compile-commands ccache \
20     && du -sh ${CCACHE_DIR} && ccache -s && rm -rf /autoware/build /autoware/log
21 ...

```

3.4 시나리오 선택

의존성까지 모두 해결이 되면 이미지가 정상적으로 생성이 됩니다. 파트별로 컨테이너를 실행하면 아래와 같이 각 파트에 정의된 패키지들이 정상적으로 설치되어 있는 것을 확인할 수 있습니다.



이제 파티셔닝의 목적에 맞는 시나리오를 선택하여 실행해 볼 수 있습니다. 시나리오 선택은 아래 내용을 기준으로 선택하였습니다.

- 파티셔닝의 목적에 부합하는 시나리오
 - Autoware를 여러 파트로 분리하여 구동시켰을때 동작 가능 여부 판단
 - 파티션간의 DDS 통신 가능 여부 판단
- 현실적으로 구동이 가능한 시나리오
 - 하드웨어가 없는 현 시점을 고려
- 문제 발생시 비교 분석이 가능하도록 기존의 **Autoware**에서 정상 동작되는 시나리오

당사는 위의 기준에 부합되는 아래 시나리오를 선택하여 테스트 하기로 결정하였습니다.

- **planning simulation**의 차선 주행 시나리오(Lane driving scenario)

3.5 파티션별 Autoware 실행하기(시나리오 실행하기)

이제 파티셔닝 환경에서 Autoware 시스템을 실행하기 위해서는 기존의 통합적인 런처 방식과는 다른 새로운 실행 방식을 설계해야 합니다. 이를 위해 크게 **파티션별 런처 구현**과 **의존성 해결**이라는 두 가지 관점을 고려해야 합니다. 이 장에서는 기존 Autoware의 런처 구조와 파티셔닝에서 발생하는 문제를 살펴보고, 각 관점에서의 해결 방안을 제안합니다. 그리고 최종적으로는 당사가 수정했던 부분에 대해서 설명을 합니다.

3.5.1 파티션별 런처 구현

Autoware는 기본적으로 **autoware_launch** 패키지를 메인 런처로 사용하는 구조로 설계되어 있습니다. 이 메인 런처는 여러 서브 런처를 실행하여 Autoware의 주요 컴포넌트를 일괄적으로 실행합니다. 하지만, 파티셔닝된 환경에서는 **autoware_launch**를 그대로 사용할 수 없는 아래와 같은 여러 가지 제약이 있습니다.

- **파티션 간 독립성 침해:**
autoware_launch는 모든 노드와 설정을 하나의 환경에서 실행한다는 전제하에 설계되었기 때문에 파티션 분리를 통해 컴포넌트를 독립적으로 실행하려는 의도와 충돌합니다.
- **의존성 문제:**
autoware_launch의 통합 실행 방식이 가지는 의미는 모든 패키지들과 의존 관계를 가진다는 의미입니다. 각 파티션에 필요한 노드가 서로 다른 컨테이너에서 실행되므로, autoware_launch의 통합 실행 방식이 작동하지 않습니다.

따라서 파티셔닝된 환경에서의 실행 방식을 설계하기 위해 몇 가지 대안을 고려했습니다:

1. ROS2 Launch 명령어를 활용한 개별 실행

- 각 파티션에서 필요한 노드를 개별적으로 실행하는 방식입니다.
- 장점:
 - 간단한 구조와 높은 유연성.
 - 별도의 런처를 설계할 필요가 없음.
- 단점:
 - 실행 과정에서 수작업이 많아 효율성이 떨어집니다.
셸 스크립트 등으로 자동화 가능하여 단점을 보완할 수 있습니다.
 - 상위 런처를 통해서 전달되었던 파라미터가 상세 분석 되어야 합니다.

2. 파티션별 맞춤형 런처 파일 작성(당사가 채용하여 적용)

- 각 파티션에 필요한 노드와 설정을 정의한 별도의 런처 파일을 작성합니다.
- 장점: 실행의 간소화와 유지보수 용이성.
- 단점: 초기 설계 작업이 필요하며, 런처 파일을 관리해야 합니다.

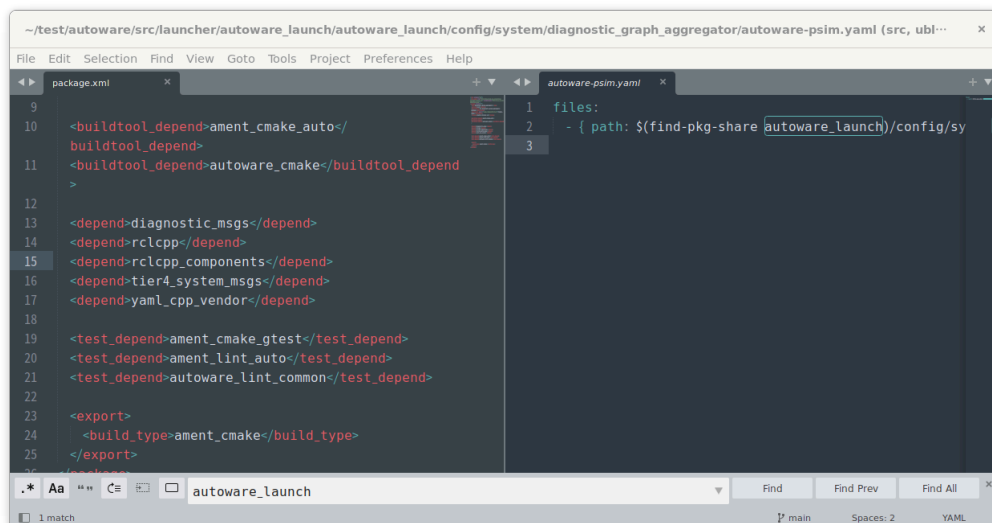
3. 기존 런처 시스템의 부분적 활용(당사가 채용하여 적용)

- 기존 런처를 최대한 활용하되, 파티션의 독립성을 유지하는 방향으로 수정합니다.
- 장점: 기존 시스템의 강점을 살리면서 파티셔닝 구조에 적응.
- 단점: 런처를 수정하면서 추가적인 테스트와 검증이 필요합니다.

당사에서는 기존 런처의 부분적 활용과 파티션별 맞춤형 런처 파일 작성을 선택했습니다. 이는 기존 런처 시스템의 안정성과 파티셔닝의 장점을 모두 살리는 타협점으로 판단되었습니다. 각 파티션별로 맞춤형 런처를 작성하면서, 기존 런처의 재사용 가능한 부분을 최대한 유지하여 개발 및 유지보수의 부담을 줄였습니다.

3.5.2 의존성 해결

위의 이미지 구성 단계, 즉 이미지를 생성하면서 파티션 빌드를 진행할때 의존성이 검출되어 해결했더라도 모든 의존성이 해결된 것은 아닙니다. 현재 Autware는 빌드 단계에서 검출되지 않는 의존성이 존재를 하며, 이는 실행할때 나타납니다. 아래와 같은 경우가 대표적인 예인데, 실제 패키지 동작시 autware_launch 를 사용하지만, package.xml 파일에는 작성된 내용이 없는 경우에 해당 됩니다.



이제 의존성 문제를 종합적으로 해결해야 하는 시점입니다. 기존 Autware는 통합된 환경에서 설계되었기 때문에, 패키지 간 의존성이 복잡하게 얹혀 있습니다. 이러한 의존성을 관리하지 않으면 실행 중 문제가 발생하거나 파티셔닝의 이점을 잃게 됩니다.

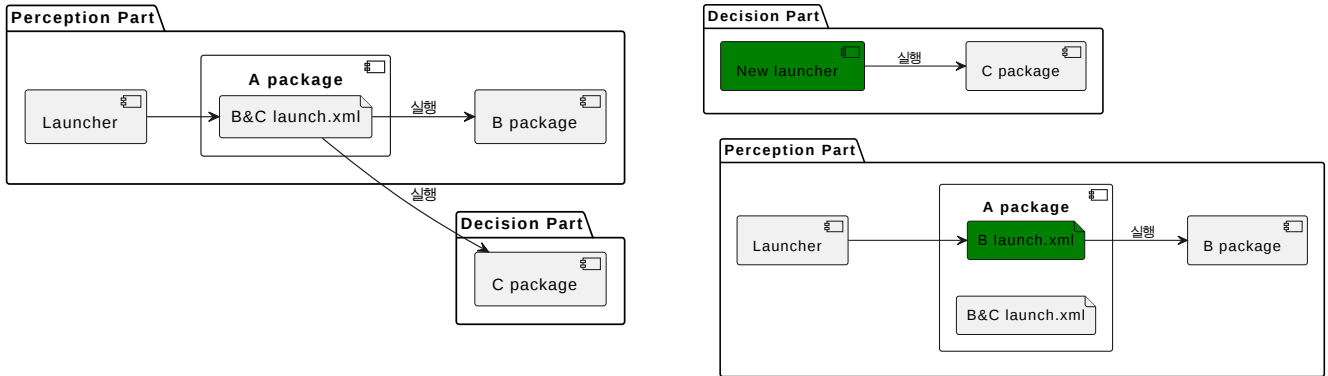
3.5.2.1 의존성 문제의 종류와 해결 방안

1. 패키지 간 실행 의존성

- 예: A 패키지가 B, C 패키지를 실행해야 하는데, C 패키지가 다른 파티션에 존재하는 경우.

○ 해결 방안:

- A 패키지와 B, C 패키지의 런처를 분리하여 각각의 파티션에서 실행되도록 구성.

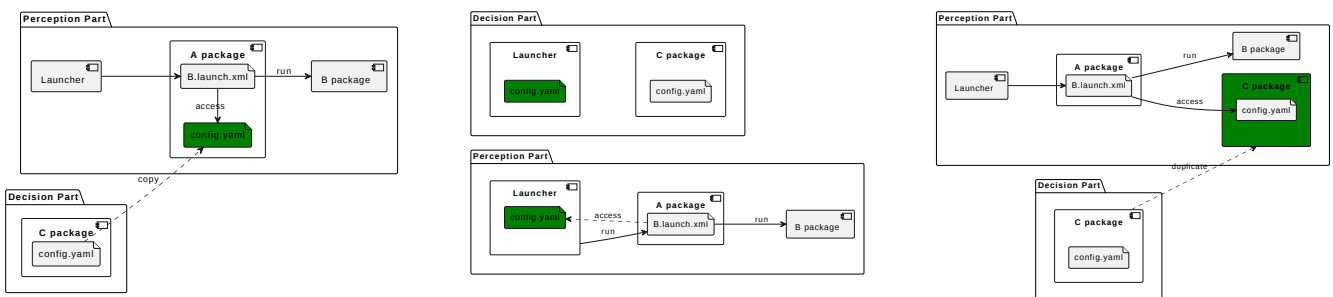
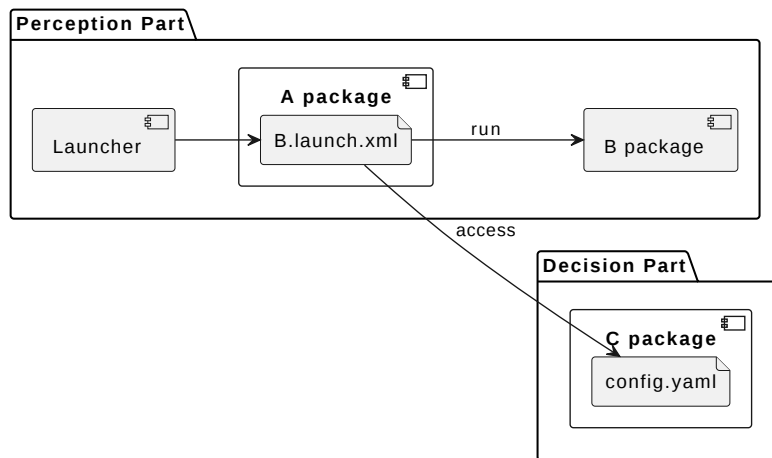


2. 설정 및 파라미터 파일 의존성

- 예: A 패키지가 실행 시 B 패키지에 존재하는 설정 파일을 참조하는 경우.

○ 해결 방안:

- 필요한 설정 파일을 A 패키지 내부로 복사하여 독립적으로 사용하도록 수정.
- 공유가 필요한 경우, 공통 데이터 영역을 구성하여 각 파티션에서 접근 가능하게 설정.
- 해당 의존 패키지를 추가하여 설정파일만 사용.



의존성을 해결하기 위해 패키지를 무작정 추가하면, 파티셔닝의 독립성과 경량화를 해칠 수 있습니다.

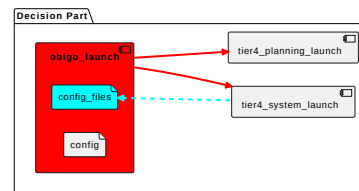
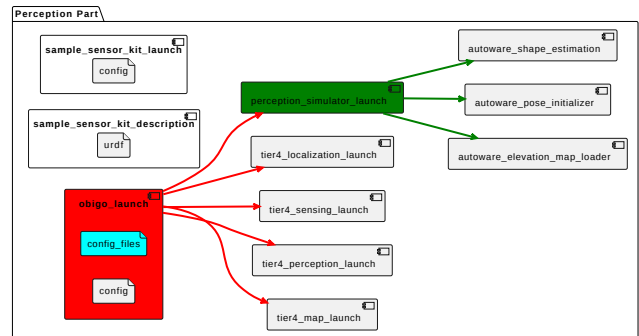
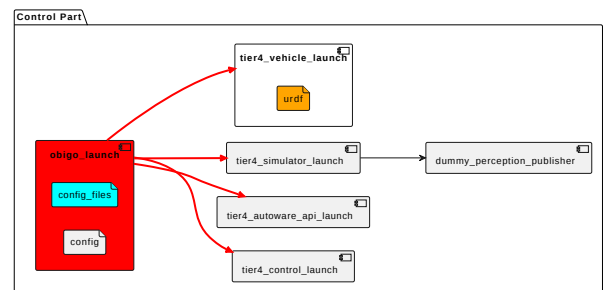
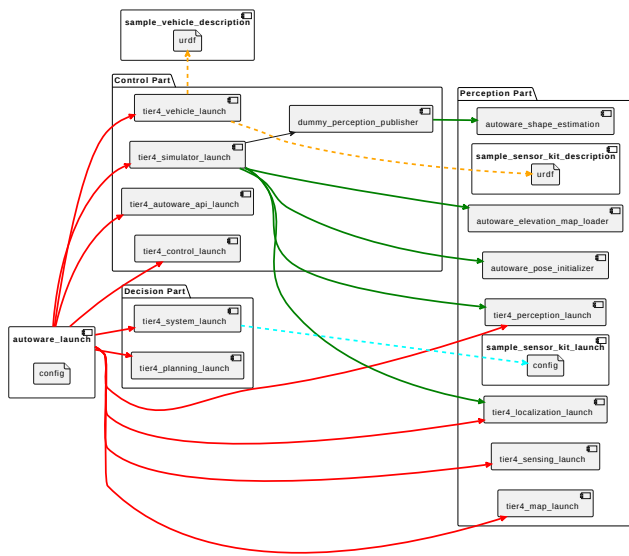
- 필수 의존성만 추가하고, 가능하면 대체 방법을 모색합니다.
- 각 파티션의 목표와 역할에 따라 의존성 범위를 최소화합니다.

3.6 수정 내용 설명

지금까지 내용을 기반으로 최종적으로는 당사가 수정했던 부분에 대해서 설명을 합니다. 파티셔닝 된 상태에서 다음 시나리오를 정상적으로 동작 시키기 위한 수정 내용입니다.

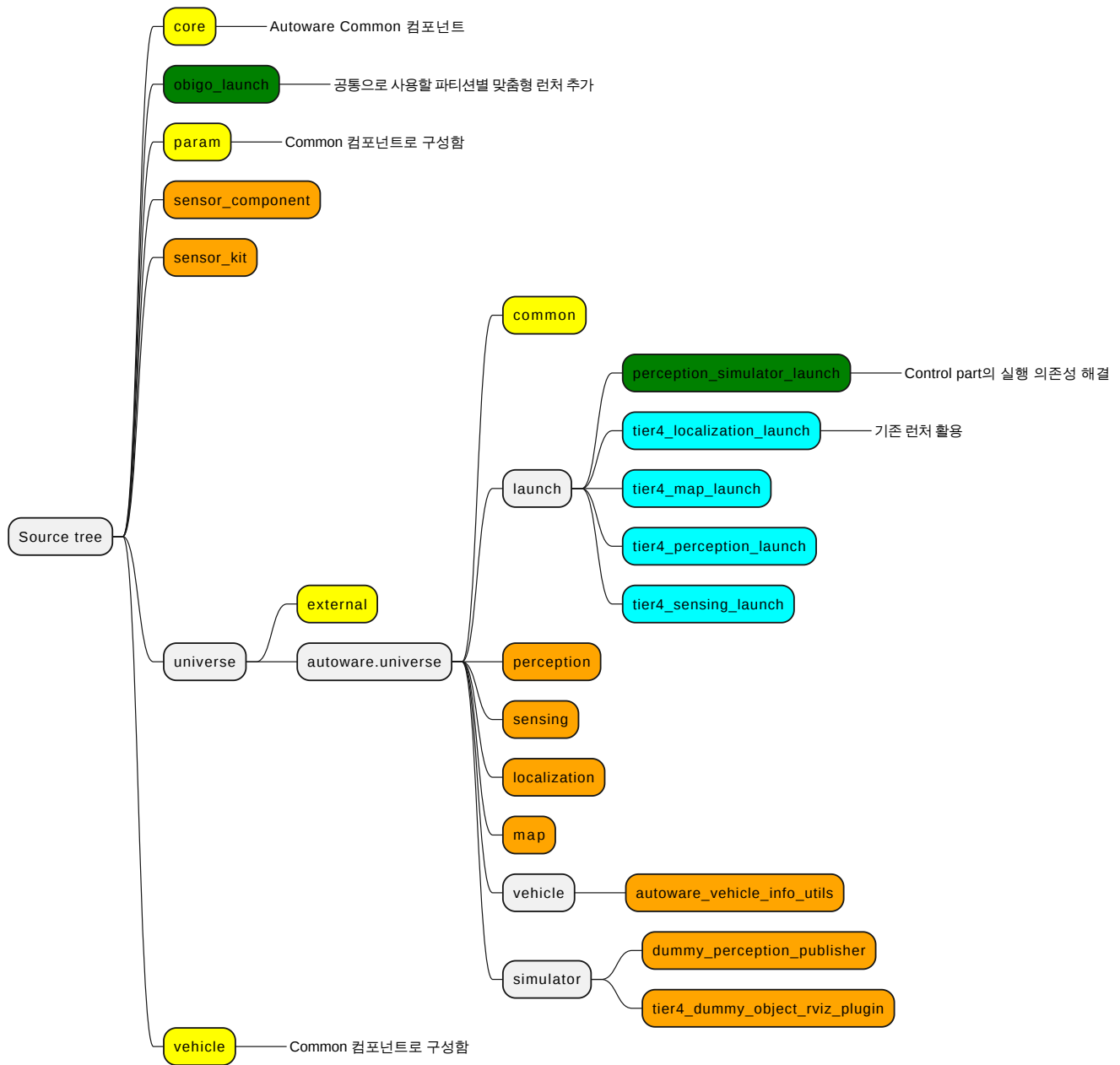
• planning simulation의 차선 주행 시나리오(Lane driving scenario)

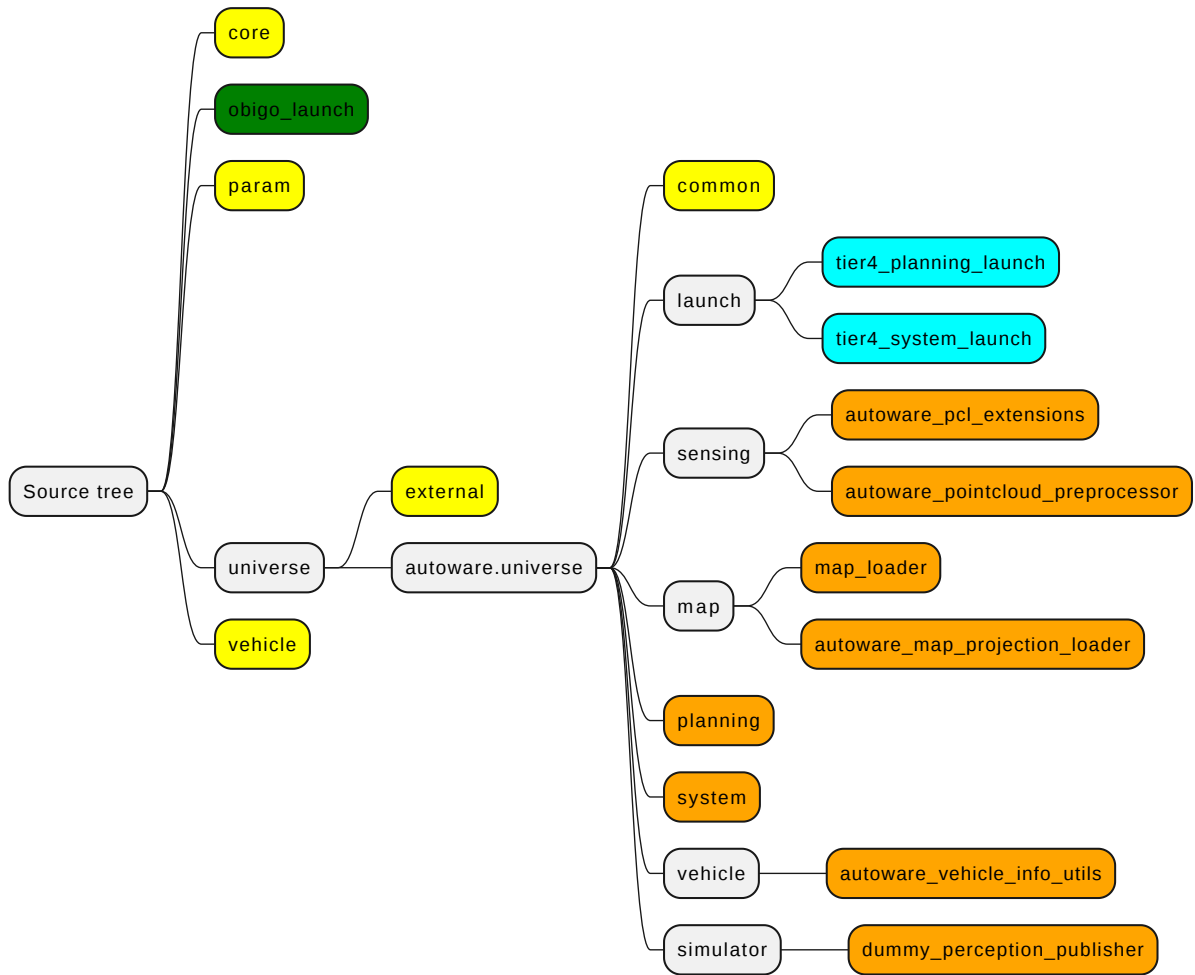
아래의 왼쪽 그림에서 파티션을 가로지르는 화살표들은 모두 의존성 문제입니다. 오른쪽 그림처럼 파티션 내부에서 독립적으로 실행되도록 의존성 문제를 해결합니다. 연관 있는 수정 내용은 같은 색으로 매핑했습니다.

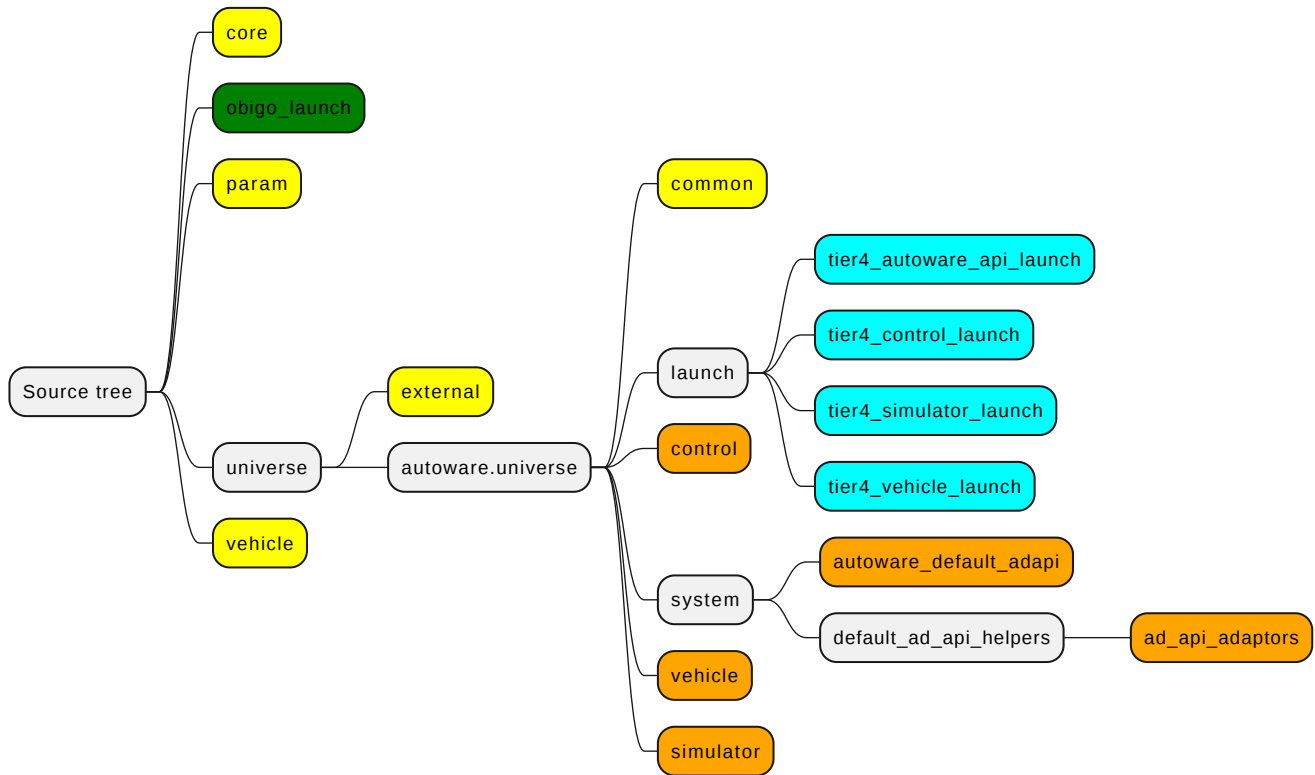


- 파티션별 맞춤형 런처 추가
 - autoware_launch 패키지 기반으로 의존성을 모두 제거한 obigo_launch 패키지를 추가(빨간색 의존성 해결)
 - Perception part에 perception_simulator_launch 패키지를 추가하여 시뮬레이션 환경의 의존성 해결(초록색 의존성 해결)
 - 각 파티션에서 obigo_launch를 사용할 수 있도록 이미지 재구성
- 공통 데이터 영역을 구성하여 각 파티션에서 접근 가능하게 설정.
 - obigo_launch 패키지의 config_files 폴더를 추가하여 설정과 파라미터 파일들을 복사(청록색 의존성 해결)
- tier4_vehicle_launch 패키지에서 필요로하는 매크로 파일을 내부로 복사하여 사용하도록 수정(주황색 의존성 해결)

아래는 파티션별 추가되는 컴포넌트와 패키지를 소스 트리 기준으로 설명합니다.







i Control Part

4. 도커 이미지 빌드하기

4.1 Autoware 다운로드

아래 명령어를 사용하여 Autoware 0.40.0 버전을 다운로드 합니다.

i `git clone -b 0.40.0 https://github.com/autowarefoundation/autoware.git`

다운로드가 완료되면 아래와 같이 autoware 폴더가 생성됩니다.


```
zeus@zeus-Super-PC: ~/main02
zeus@zeus-Super-PC:~/main02$ ll
합계 8
drwxrwxr-x 2 zeus zeus 4096 1월 13 16:35 ./
drwxr-x--- 41 zeus zeus 4096 1월 13 16:33 ../
zeus@zeus-Super-PC:~/main02$ git clone -b 0.40.0 https://github.com/autwarefoundation/autware.git
'autware'에 복제합니다...
remote: Enumerating objects: 4134, done.
remote: Counting objects: 100% (308/308), done.
remote: Compressing objects: 100% (179/179), done.
remote: Total 4134 (delta 255), reused 130 (delta 129), pack-reused 3826 (from 5)
오브젝트를 받는 중: 100% (4134/4134), 14.01 MiB | 18.78 MiB/s, 완료.
델타를 알아내는 중: 100% (2449/2449), 완료.
Note: switching to '8c66826b04b98d363effd6f6c6df84a0cc092ef'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

zeus@zeus-Super-PC:~/main02$ ll
합계 12
drwxrwxr-x 3 zeus zeus 4096 1월 13 16:35 ./
drwxr-x--- 41 zeus zeus 4096 1월 13 16:33 ../
drwxrwxr-x 8 zeus zeus 4096 1월 13 16:36 autware/
zeus@zeus-Super-PC:~/main02$
```

i 호스트 PC에 git 환경이 구축되어 있어야 합니다. 또는 zip 파일로 다운받아 사용 가능합니다.

4.2 이미지 빌드하기

도커 이미지는 기본적으로 아래 스크립트를 사용하여 빌드할 수 있습니다.

- ./docker/build.sh - Autoware에서 제공하는 빌드 스크립트. Autoware가 하나의 이미지에 모두 포함됨.
- ./docker/partition/partition_build.sh - 파티셔닝을 위해서 작업된 파티션별 이미지를 생성하는 빌드 스크립트. 추후 패치 예정.

아래와 같이 x86_64와 arm64 아키텍처용 이미지를 빌드할 수 있습니다. 디폴트는 'linux/amd64' 입니다.

- x86_64 아키텍처용 이미지 빌드
 - ./docker/build.sh
 - ./docker/build.sh --platform linux/amd64
- arm64 아키텍처용 이미지 빌드
 - ./docker/build.sh --platform linux/arm64

i 스크립트가 정상적으로 동작하려면 호스트 PC에 vcs tool, docker 환경이 구축되어 있어야 합니다.

vcs tool은 실제 Autoware 소스코드를 다운받기 위해서 사용하는 툴이고, docker는 이미지를 빌드하는데 사용합니다.

docker는 Autoware 다운로드 후 './setup-dev-env.sh -y docker' 명령어로도 설치가 가능합니다.

해당 명령어는 도커 엔진 뿐만 아니라, cuda, nvidia_container_toolkit 도 함께 설치할 수 있어 사용을 권장합니다.

4.3 ARM 빌드 환경 설정하기

ARM용 도커 이미지 빌드를 진행하다 보면 아래와 같이 'exec format error' 에러가 발생합니다.

```
zeus@zeus-Super-PC: ~/main02/autoware

#6 extracting sha256:ebaafba2dac67131d8abb8cc1c154deefa1f261c8a6ea45890c38a327ab8ecba done
#6 extracting sha256:8bc16a1d1ac077be73e099a09b28bd4a72f6e448919e903104f9f48a9ccdd13d 0.1s
#6 extracting sha256:8bc16a1d1ac077be73e099a09b28bd4a72f6e448919e903104f9f48a9ccdd13d 0.8s done
#6 DONE 10.5s

#8 [base-cuda base 2/13] COPY setup-dev-env.sh ansible-galaxy-requirements.yaml amd64.env arm64.env /autoware/
#8 DONE 0.7s

#9 [base base 3/13] COPY ansible/ /autoware/ansible/
#9 DONE 0.1s

#10 [base base 4/13] COPY docker/scripts/cleanup Apt.sh /autoware/cleanup Apt.sh
#10 DONE 0.1s

#11 [base base 5/13] RUN chmod +x /autoware/cleanup Apt.sh
#11 0.148 exec /bin/bash: exec format error
#11 ERROR: process "/bin/bash -o pipefail -c chmod +x /autoware/cleanup Apt.sh" did not complete successfully: exit code: 1
.....
> [base-cuda base 5/13] RUN chmod +x /autoware/cleanup Apt.sh:
0.148 exec /bin/bash: exec format error
.....

1 warning found (use docker --debug to expand):
 - InvalidDefaultArgInFrom: Default value for ARG $BASE_IMAGE results in empty or invalid base image name (line 4)
Dockerfile.base:12
-----
10 | COPY ansible/ /autoware/ansible/
11 | COPY docker/scripts/cleanup Apt.sh /autoware/cleanup Apt.sh
12 | >>> RUN chmod +x /autoware/cleanup Apt.sh
13 | COPY docker/scripts/cleanup system.sh /autoware/cleanup system.sh
14 | RUN chmod +x /autoware/cleanup system.sh
-----
ERROR: target base: failed to solve: process "/bin/bash -o pipefail -c chmod +x /autoware/cleanup Apt.sh" did not complete successfully: exit code: 1
zeus@zeus-Super-PC: ~/main02/autoware$
```

아래와 같이 도커의 빌더가 지원하는 플랫폼을 확인해 보면 arm64가 존재하지 않습니다.

```
zeus@zeus-Super-PC: ~/main02/autoware

zeus@zeus-Super-PC:~/main02/autoware$ docker buildx inspect
Name:          default
Driver:        docker
Last Activity: 2025-01-13 06:49:19 +0000 UTC

Nodes:
Name:          default
Endpoint:      default
Status:        running
BuildKit version: v0.16.0
Platforms:     linux/amd64, linux/amd64/v2, linux/amd64/v3, linux/386
Labels:
org.mobyproject.buildkit.worker.moby.host-gateway-ip: 172.17.0.1
zeus@zeus-Super-PC:~/main02/autoware$
```

아래 명령어로 QEMU를 설치하여 멀티 아키텍처 빌드가 지원되도록 환경을 설정합니다.

i `docker run --rm --privileged multiarch/qemu-user-static --reset -p yes`

```
zeus@zeus-Super-PC: ~/main02/autoware

zeus@zeus-Super-PC:~/main02/autoware$ docker buildx inspect
Name:          default
Driver:        docker
Last Activity: 2025-01-13 07:48:20 +0000 UTC

Nodes:
Name:          default
Endpoint:      default
Status:        running
BuildKit version: v0.16.0
Platforms:     linux/amd64, linux/amd64/v2, linux/amd64/v3, linux/386, linux/arm64, linux/riscv64, linux/ppc64, linux/ppc64le, linux/s390x, linux/mips64le,
linux/mips64, linux/arm/v7, linux/arm/v6
Labels:
org.mobyproject.buildkit.worker.moby.host-gateway-ip: 172.17.0.1
zeus@zeus-Super-PC:~/main02/autoware$
```

5. 개발 및 테스트 환경

5.1 하드웨어 사양

- 프로세서 (CPU): AMD Ryzen 9 5950X 16-Core Processor (기본 클럭 3.4GHz, 최대 부스트 클럭 4.9GHz)

- 메모리 (RAM): 64GB DDR4
- 그래픽카드 (GPU): NVIDIA GeForce RTX 3060
- 저장공간: SSD 300GB

```

zeus@zeus-Super-PC: ~
zeus@zeus-Super-PC:~$
zeus@zeus-Super-PC:~$ uname -a
Linux zeus-Super-PC 6.8.0-48-generic #48-22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Mon Oct 7 11:24:13 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
zeus@zeus-Super-PC:~$
zeus@zeus-Super-PC:~$ sudo dmidecode -t processor | grep -E "(Version|Core Count|Core Enabled|Thread Count|Max|Current)"
[sudo] zeus 암호 :
Version: AMD Ryzen 9 5950X 16-Core Processor
Max Speed: 5950 MHz
Current Speed: 3400 MHz
Core Count: 16
Core Enabled: 16
zeus@zeus-Super-PC:~$
zeus@zeus-Super-PC:~$ free -g
              총 계      사용         여분      공유      버퍼/캐시      가용
메 모 리 :         62         11          3          0         47         49
스 랍 :             0           0           0
zeus@zeus-Super-PC:~$
zeus@zeus-Super-PC:~$ nvidia-smi
Tue Nov 26 14:19:21 2024

+-----+
| NVIDIA-SMI 555.42.06              Driver Version: 555.42.06   CUDA Version: 12.5   |
+-----+-----+
| GPU Name      Persistence-M | Bus-Id  Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0  NVIDIA GeForce RTX 3060      Off | 00000000:07:00.0 Off |          N/A         |
| 0%   35C   P8      11W / 180W | 28M1B / 12288M1B |      0%   Default   |
+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI        PID   Type   Process name                  GPU Memory |
| ID    ID                          |                     | Usage      |
+-----+-----+
| 0   N/A  N/A       1452    G   /usr/lib/xorg/Xorg              9M1B      |
| 0   N/A  N/A       1932    G   /usr/bin/gnome-shell            3M1B      |
+-----+-----+
zeus@zeus-Super-PC:~$

```

5.2 소프트웨어 환경

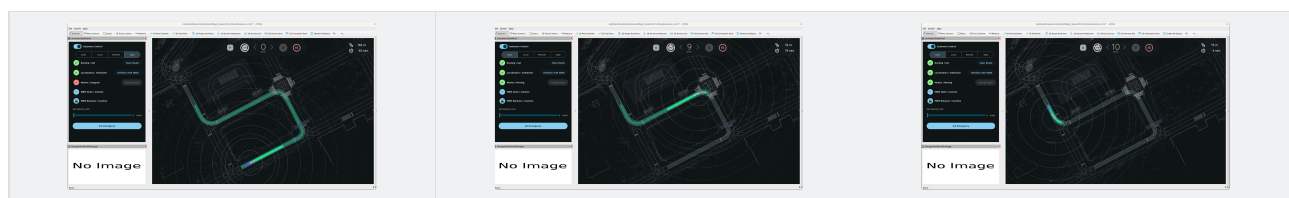
- 운영체제 (OS): Ubuntu 22.04 LTS
- 오토웨어 버전: Autoware humble (git clone -b humble <https://github.com/autowarefoundation/autoware.git>)
- 네트워크: 테스트 환경은 오프라인 모드로 실행되었으며, 시뮬레이션 데이터는 로컬에서 관리.

5.3 시뮬레이션 환경

- 시뮬레이션 도구: AD Api 기반의 Rviz2 시뮬레이터와 연동.
- 맵 데이터: sample-map-planning.zip(https://docs.google.com/uc?export=download&id=1499_nsbUbleturZaDj7jhUownh5fvXHd)
- 시뮬레이션 조건: Autoware planning simulation의 차선 주행 시나리오(Lane driving scenario)

6. 테스트 방법

1. 파티션별 도커 이미지 실행.
2. Rviz2 툴에서 초기 포즈와 목표 포즈 설정
3. Rviz2 툴에서 Auto 모드 실행
4. 차량이 정해진 차선을 따라 자율 주행되는 확인



File	Modified
>  Dockerfile.svg	Oct 07, 2024 by zeus.yoon
>  image-20241126-052108.png	Nov 26, 2024 by zeus.yoon
>  ad01.png	Nov 26, 2024 by zeus.yoon
>  ad02.png	Nov 26, 2024 by zeus.yoon
>  ad03.png	Nov 26, 2024 by zeus.yoon
>  GOASP_OSS_1세부_멀티 OS 지원 Linux_검증 보고서_V0.0.1.pptx	Nov 26, 2024 by William Woo
>  스크린샷 2024-11-26 15-22-19.png	Nov 26, 2024 by zeus.yoon
>  스크린샷 2024-11-26 15-21-53.png	Nov 26, 2024 by zeus.yoon
>  스크린샷 2024-11-26 15-22-40.png	Nov 26, 2024 by zeus.yoon
>  스크린샷 2024-11-26 15-23-11.png	Nov 26, 2024 by zeus.yoon
>  image-20241203-033254.png	Dec 03, 2024 by zeus.yoon
>  image-20241203-033333.png	Dec 03, 2024 by zeus.yoon
>  image-20241203-053419.png	Dec 03, 2024 by zeus.yoon
>  image-20241203-055044.png	Dec 03, 2024 by zeus.yoon
>  image-20241203-063057.png	Dec 03, 2024 by zeus.yoon
>  image-20250113-073657.png	Jan 13, 2025 by zeus.yoon
>  image-20250113-074555.png	Jan 13, 2025 by zeus.yoon
>  image-20250113-075049.png	Jan 13, 2025 by zeus.yoon
>  image-20250113-084125.png	Jan 13, 2025 by zeus.yoon

 Drag and drop to upload or [browse for files](#) 

 [Download All](#)